ML Experiment Number 10

Student Name : Varsharani Pavankumar Kasliwal

Batch :CE01

Roll Number :CE14

Aim

To implement DBSCAN clustering using Python for customer segmentation on the Mall Customer Segmentation dataset and identify arbitrary-shaped clusters along with spatial outliers (noise).

Objectives

1. To understand the principles of Density-Based Spatial Clustering of Applications with Noise (DBSCAN).
2. To implement DBSCAN clustering using Python.3. To preprocess and feature-scale tabular customer data using StandardScaler.
3. To determine optimal hyperparameters (ε and $min_samples$) using k-distance graph analysis.
4. To identify core points, border points, and spatial outliers (noise points denoted by label -1).
5. To evaluate clustering performance using metrics like Silhouette Score.

Relevant Theory

###DBSCAN (Density-Based Spatial Clustering of Applications with Noise) DBSCAN is an unsupervised non-parametric clustering algorithm that groups data points based on local density estimations. Unlike centroid-based algorithms (e.g., K-Means), DBSCAN can discover arbitrary-shaped clusters and automatically isolate noisy data points without requiring the user to specify the total number of clusters in advance.

Core Concepts and Parameters:

1. ε (Epsilon): The maximum radius of the neighborhood around a given data point x.
2. $min_samples$ (MinPts): The minimum number of points within the ε-neighborhood required to form a dense region.

Point ClassificationsCore Point:

- A point that has at least $min_samples$ within its ε-neighborhood.

- Border Point: A point that is within the ε-neighborhood of a Core Point but has fewer than $min_samples$ in its own neighborhood.
- Noise Point (Outlier): A point that is neither a Core Point nor a Border Point. Noise points are assigned a cluster label of -1.

###Dataset Information Get the Mall Customer Segmentation dataset from Kaggle using the following link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

The dataset contains spending patterns and demographic attributes of mall customers:

- CustomerID: Unique identifier for each customer.
- Gender: Male or Female.
- Age: Age of the customer.
- Annual Income (k$): Annual income of the customer in thousands of dollars.
- Spending Score (1-100): Score assigned based on customer behavior and purchasing patterns.

For clustering, we will use: Annual Income (k$) and Spending Score (1-100).

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).
-

Load the Mall Customer Segmentation dataset (Mall_Customers.csv).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

# Load dataset
df = pd.read_csv('/content/Mall_Customers.csv')
```

```
# Display the first few rows of the dataset
display(df.head())
print(df.info())
```

{"summary":"{\n \"name\": \"print(df\",\n \"rows\": 5,\n \"fields\": [\n   {\n   \"co

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
```

Task 2: Feature Selection and Standardization

- Select target features (Annual Income (k) and Spending Score (1-100)).
- Scale features using Standard Scaler to bring feature dimensions to a comparable range ($\mu = 0, \sigma = 1$), which is crucial for distance-based neighborhood computations.

```
# Select target features
features = ['Annual Income (k$)', 'Spending Score (1-100)']
X = df[features].values

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Shape of scaled features:", X_scaled.shape)
print("Mean of features after scaling:", np.round(X_scaled.mean(axis=0), 2))
print("Std of features after scaling:", X_scaled.std(axis=0))
```

```
Shape of scaled features: (200, 2)
Mean of features after scaling: [-0. -0.]
Std of features after scaling: [1. 1.]
```

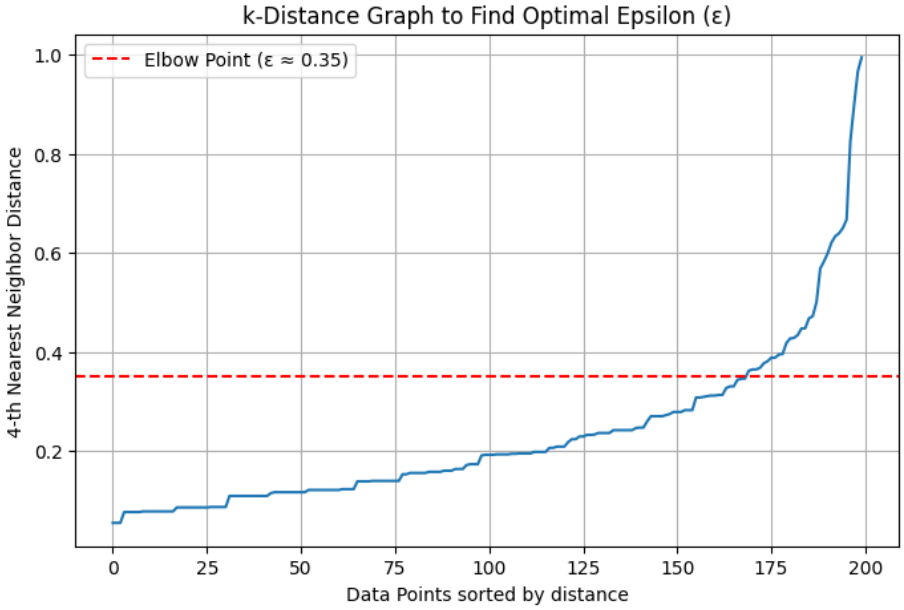
Task 3: Hyperparameter Selection via k-Distance Graph

- Compute Nearest Neighbors using NearestNeighbors ($k = min_samples$).
- Plot sorted k-distances to locate the "elbow point," which indicates the optimal value for ε (Epsilon).

```
# Using k = 4 (as min_samples is typically chosen >= dimensions + 1; here 2D features -> Mi
min_samples = 4
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

# Sort distance to the kth nearest neighbor
sorted_distances = np.sort(distances[:, min_samples - 1])

# Plot the k-distance graph
plt.figure(figsize=(8, 5))
plt.plot(sorted_distances)
plt.title('k-Distance Graph to Find Optimal Epsilon (\u03b5)')
plt.xlabel('Data Points sorted by distance')
plt.ylabel(f'{min_samples}-th Nearest Neighbor Distance')
plt.grid(True)
plt.axhline(y=0.35, color='r', linestyle='--', label='Elbow Point (\u03b5 \u2248 0.35)')
plt.legend()
plt.show()
```



Task 4: Implement DBSCAN Clustering

- Instantiate sklearn.cluster.DBSCAN with the identified ε and *min_samples*.
- Fit the model on scaled data to generate cluster assignments and noise labels (-1).

```
# We select epsilon = 0.35 based on the elbow point from the k-distance graph
epsilon = 0.35
min_samples = 4

dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
dbscan.fit(X_scaled)

# Append labels back to original dataframe
df['Cluster'] = dbscan.labels_

print("DBSCAN labels generated:", np.unique(dbscan.labels_))
```

```
DBSCAN labels generated: [-1  0  1  2  3  4  5  6]
```

Task 5: Evaluate DBSCAN Clustering

- Cluster Count: Calculate the number of clusters generated (excluding noise points).
- Noise Point Analysis: Count and evaluate identified spatial outliers (label -1).
- Silhouette Score: Calculate Silhouette Score to evaluate cluster cohesion and separation.

```
labels = dbscan.labels_

# Calculate number of clusters (excluding noise points label -1)
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
n_noise_ = list(labels).count(-1)

print(f'Estimated number of clusters: {n_clusters_}')
print(f'Estimated number of noise points: {n_noise_}')

# Calculate Silhouette Score
if n_clusters_ > 1:
    silhouette_avg = silhouette_score(X_scaled, labels)
    print(f"Silhouette Score: {silhouette_avg:.3f}")
else:
    print("Silhouette Score cannot be computed (less than 2 clusters found).")
```

```
Estimated number of clusters: 7
Estimated number of noise points: 16
Silhouette Score: 0.426
```

Task 6: Analyze Results and Cluster Statistics

- Extract cluster labels and append them back to the original dataset.
- Calculate mean statistics (e.g., average Annual Income and Spending Score) per cluster to interpret final customer segments and evaluate detected noise points.

```
# Analyze Cluster Statistics
print("--- Cluster Statistics (Mean Values) ---")
cluster_stats = df.groupby('Cluster')[['Age', 'Annual Income (k$)', 'Spending Score (1-100)'
display(cluster_stats)

print("\n--- Visualizing Clusters and Noise ---")
plt.figure(figsize=(10, 6))

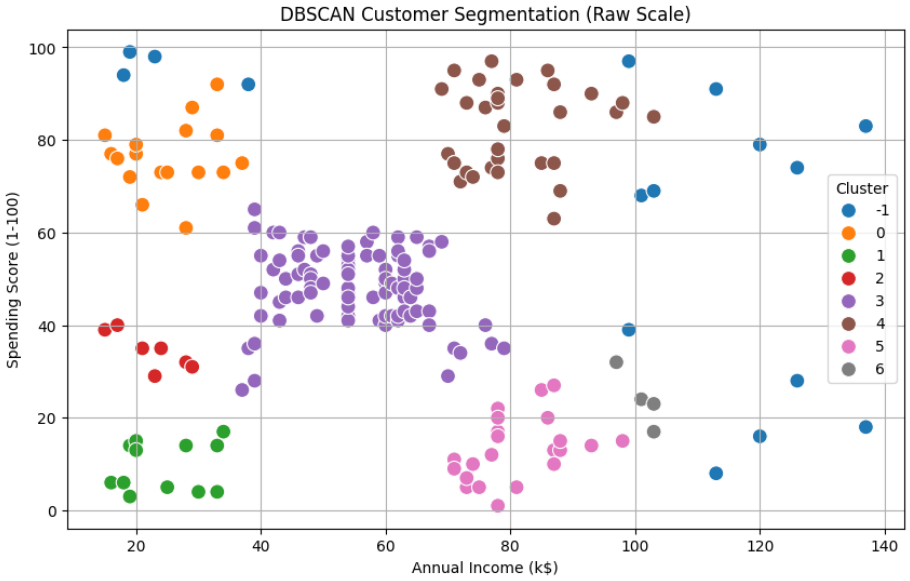
# Generate color palette
```

```
sns.scatterplot(
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='tab10',
    data=df,
    legend='full',
    s=100
)
plt.title('DBSCAN Customer Segmentation (Raw Scale)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.grid(True)
plt.show()
```

```
--- Cluster Statistics (Mean Values) ---
```


```
{"summary":"{\n  \"name\": \"cluster_stats\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n
```


```
--- Visualizing Clusters and Noise ---
```



Conclusion

The experiment successfully implemented DBSCAN clustering on the Mall Customer Segmentation dataset. Feature standardization ensured balanced neighborhood distance calculations across income and spending metrics. By

analyzing k-distance graphs, optimal ε and *min_samples* parameters were selected, enabling the discovery of density-defined customer segments and explicit identification of spatial noise points (outliers) without requiring a pre-defined cluster count.